

```

#CODE WITH KNN,NCF,RNN AND RL (updated) with the corelation map part
import streamlit as st
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import random
import os
import urllib.parse

# from sklearn.neighbors import NearestNeighbors

st.set_page_config(page_title="MRS", layout="wide")

st.markdown("""
<style>
/* Top Streamlit header bar */
header[data-testid="stHeader"] {
    background: linear-gradient(135deg, #0f3d3e, #145c5f);
}

/* Toolbar (Deploy, menu dots) */
header[data-testid="stHeader"] * {
    color: #e6fffa !important;
}

/* Main app background */
.stApp {
    background: linear-gradient(135deg, #0f3d3e, #1f7a6d, #2fa4a9);
    color: #e6fffa;
}

/* Remove white gap under header */
div[data-testid="stAppViewContainer"] {
    background: transparent;
}
</style>
""", unsafe_allow_html=True)

# -----
# Load Dataset
# -----
@st.cache_data
def load_data():
    df = pd.read_csv("Music_dataset2.csv")

    # Normalize column names
    df.columns = df.columns.str.strip().str.lower()

```

```

# Robust renaming for artist column
df.rename(columns={
    "track name": "song",
    "track": "song",
    "artist name(s)": "artist",
    "artist name": "artist",
    "artists": "artist",
    "artist": "artist",
    "genres": "genre",
    "release date": "release_date",
    "release_date": "release_date"
}, inplace=True)

if "artist" not in df.columns:
    df["artist"] = "Unknown Artist"

# Convert release date to datetime and extract year
if "release_date" in df.columns:
    df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
    df["year"] = df["release_date"].dt.year
else:
    df["year"] = np.nan

if "genre" not in df.columns:
    df["genre"] = "Unknown"
required_cols = [c for c in ["song", "artist"] if c in df.columns]
df.dropna(subset=required_cols, inplace=True)
df.reset_index(drop=True, inplace=True)

# Ensure audio features are float and clipped to 0–1
for col in ["valence", "energy", "tempo"]:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0.5)
        if col != "tempo": # tempo is 0–250 BPM, not 0–1
            df[col] = df[col].clip(0.0, 1.0)

return df

df = load_data()

# -----
# Mood–HRV–Stress Alignment Function (BIAS-FREE VERSION)
# Uses audio features instead of genre string matching
# -----

MOOD_AUDIO_TARGETS = {
    "Sad":    {"valence": (0.0, 0.35), "energy": (0.0, 0.40)},
    "Calm":  {"valence": (0.3, 0.60), "energy": (0.0, 0.35)},

```

```

    "Energetic": {"valence": (0.5, 1.00), "energy": (0.7, 1.00)},
    "Angry":     {"valence": (0.0, 0.40), "energy": (0.6, 1.00)},
    "Happy":     {"valence": (0.6, 1.00), "energy": (0.4, 0.80)},
}

def mood_physiology_fit(row, mood_state, hrv, stress):
    score = 0.0

    # — Audio feature matching (culture-neutral) —————
    target = MOOD_AUDIO_TARGETS.get(mood_state, {})
    for feature, (low, high) in target.items():
        val = row.get(feature, None)
        if val is not None and low <= val <= high:
            score += 1.0

    # — HRV adjustment (high HRV → prefer calmer audio) —————
    energy = row.get("energy", None)
    if energy is not None:
        if hrv > 80 and energy < 0.4:
            score += 0.4
        elif hrv < 50 and energy > 0.6:
            score += 0.4

    # — Stress adjustment (high stress → prefer low energy) ———
    if stress > 60:
        if energy is not None and energy < 0.4:
            score += 0.3
        if energy is not None and energy > 0.7:
            score -= 0.3

    return score

# -----
# Load Training Metadata (VERY IMPORTANT)
# -----
# ----- Safe metadata loading -----
try:
    metadata = torch.load("metadata2.pth", map_location="cpu")

    genre_mapping = metadata["genre_mapping"]
    vibe_mapping  = metadata["vibe_mapping"]

except Exception as e:
    st.error("Metadata loading failed. Please retrain models.")
    st.stop()

NUM_SONGS_TRAINED = metadata["num_songs"]
NUM_GENRES_TRAINED = metadata["num_genres"]

```

```

NUM_VIBES_TRAINED = metadata["num_vibes"]

genre_mapping = metadata["genre_mapping"]
vibe_mapping = metadata["vibe_mapping"]

# -----
# Ensure REQUIRED columns exist (CRITICAL)
# -----

# Genre safety
if "genre" not in df.columns:
    df["genre"] = "Unknown"
df["genre"] = df["genre"].fillna("Unknown")

# Vibe safety (DATASET DOES NOT HAVE IT → CREATE)
if "vibe" not in df.columns:
    df["vibe"] = "Neutral"
df["vibe"] = df["vibe"].fillna("Neutral")

# ----- Load Trained Models -----
num_songs = len(df)

# ----- RNN -----
class ContextRNN(nn.Module):
    def __init__(self, num_songs, num_genres, num_vibes, embed_dim=128,
hidden_dim=64):
        super().__init__()
        self.song_emb = nn.Embedding(num_songs, embed_dim)
        self.genre_emb = nn.Embedding(num_genres, 8)
        self.vibe_emb = nn.Embedding(num_vibes, 8)
        self.context_fc = nn.Linear(6, 16) # mood, stress, hrv
        self.lstm = nn.LSTM(160, 64, batch_first=True)
        self.dropout = nn.Dropout(0.3)
        self.attention = nn.Linear(64, 1)
        self.fc = nn.Linear(64, num_songs)

    def forward(self, seq, genre, vibe, ctx):
        s = self.song_emb(seq)
        g = self.genre_emb(genre).unsqueeze(1).repeat(1, seq.size(1), 1)
        v = self.vibe_emb(vibe).unsqueeze(1).repeat(1, seq.size(1), 1)
        c = self.context_fc(ctx).unsqueeze(1).repeat(1, seq.size(1), 1)
        x = torch.cat([s, g, v, c], dim=2)
        out, _ = self.lstm(x)
        out = self.dropout(out)
        attn_weights = torch.softmax(self.attention(out), dim=1) #Attention weights
        context = torch.sum(attn_weights * out, dim=1) # Weighted context vector
        return self.fc(context)

```

```

# ----- NCF -----
class ContextNCF(nn.Module):
    def __init__(self, num_users, num_songs, num_genres, num_vibes):
        super().__init__()
        self.user_emb = nn.Embedding(num_users, 16) # user embedding
        self.song_emb = nn.Embedding(num_songs, 32)
        self.genre_emb = nn.Embedding(num_genres, 8)
        self.vibe_emb = nn.Embedding(num_vibes, 8)
        self.mood_emb = nn.Embedding(6, 4)

        total_input = 16 + 32 + 8 + 8 + 4 + 1 + 1 + 1 + 1 + 1 + 1 # 16 from user

        self.fc1 = nn.Linear(total_input, 64)
        self.fc2 = nn.Linear(64, 32)
        self.out = nn.Linear(32, 1)

    def forward(self, u, s, g, v, m, st, h, t, d, w, pop):
        x = torch.cat([
            self.user_emb(u),
            self.song_emb(s),
            self.genre_emb(g),
            self.vibe_emb(v),
            self.mood_emb(m),
            st,
            h,
            t,
            d,
            w,
            pop
        ], dim=1)

        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.out(x)

# ----- Load Trained Models -----
# From trained_model.py
num_songs = len(df)

# Ensure 'genre' column exists
if "genre" in df.columns:
    df["genre"] = df["genre"].fillna("Unknown")
else:
    df["genre"] = "Unknown"

# Ensure 'vibe' column exists
if "vibe" in df.columns:
    df["vibe"] = df["vibe"].fillna("Neutral")

```

```

else:
    df["vibe"] = "Neutral"

# -----
# FINAL SAFE MAPPING (NO OVERFLOW POSSIBLE)
# -----
df["genre_id"] = (
    df["genre"]
    .map(genre_mapping)
    .fillna(0)
    .astype(int)
    .clip(0, NUM_GENRES_TRAINED - 1)
)

df["vibe_id"] = (
    df["vibe"]
    .map(vibe_mapping)
    .fillna(0)
    .astype(int)
    .clip(0, NUM_VIBES_TRAINED - 1)
)

# Load trained models with original dimensions
num_songs_trained = len(df)
num_genres_trained = len(genre_mapping)
num_vibes_trained = len(vibe_mapping)

assert df["genre_id"].max() < num_genres_trained #Genre ID exceeds trained embedding
size
assert df["vibe_id"].max() < num_vibes_trained #Vibe ID exceeds trained embedding size

rnn_model = ContextRNN(
    num_songs=NUM_SONGS_TRAINED,
    num_genres=NUM_GENRES_TRAINED,
    num_vibes=NUM_VIBES_TRAINED
)

NUM_USERS_TRAINED = metadata["num_users"] # <-- number of users in your training
dataset
ncf_model = ContextNCF(
    num_users=NUM_USERS_TRAINED,
    num_songs=NUM_SONGS_TRAINED,
    num_genres=NUM_GENRES_TRAINED,
    num_vibes=NUM_VIBES_TRAINED
)

# Load trained weights

```

```

rnn_model.load_state_dict(torch.load("rnn_model_trained2.pth", map_location="cpu"),
strict=True)
ncf_model.load_state_dict(torch.load("ncf_model_trained2.pth",
map_location="cpu"),strict=True)

rnn_model.eval()
ncf_model.eval()

# -----
# Song Mapping (VERY IMPORTANT)
# -----
df["song_id"] = df.index.astype(int)

# -----
# KNN Feature Preparation
# -----
# def build_knn_features(df):
#     df_knn = df.copy()

#     # Genre encoding (very simple & robust)
#     def genre_encode(g):
#         g = str(g).lower()
#         if "ghazal" in g or "classical" in g or "raga" in g:
#             return 0
#         if "bollywood" in g:
#             return 1
#         if "pop" in g:
#             return 2
#         return 3

#     df_knn["genre_enc"] = df_knn["genre"].apply(genre_encode)

#     # Year normalization
#     df_knn["year_norm"] = df_knn["year"].fillna(df_knn["year"].median())
#     df_knn["year_norm"] = (df_knn["year_norm"] - df_knn["year_norm"].min()) / (
#         df_knn["year_norm"].max() - df_knn["year_norm"].min() + 1e-6
#     )

#     return df_knn[["genre_enc", "year_norm"]].values

knn_features = build_knn_features(df)

# # Train KNN model (ONCE)
# knn_model = NearestNeighbors(
#     n_neighbors=50,
#     metric="cosine"
# )

```

```

# knn_model.fit(knn_features)

num_songs = len(df)

# -----
# RL Agent
# -----
class RLAgent:
    def __init__(self, n_actions):
        self.alpha = 0.2
        self.gamma = 0.9
        self.q_table = np.zeros((100, n_actions))

    def update_q(q, s, a, r, ns, alpha=0.1, gamma=0.9):
        q[s, a] += alpha * (r + gamma * np.max(q[ns]) - q[s, a])

# -----
# Spotify Link
# -----
def spotify_link(song, artist):
    song = song if pd.notna(song) else ""
    artist = artist if pd.notna(artist) else ""
    q = urllib.parse.quote_plus(f"{song} {artist}")
    return f"https://open.spotify.com/search/{q}"

# =====
# MAIN PAGE INPUTS
# =====
st.title("🎧 Music Recommendation System") #music recommendation system

# -----
# User Details
# -----
st.header("👤 User Details")

col1, col2, col3 = st.columns([1,1,1])
with col1:
    name = st.text_input("Name", "Guest")

# ----- AGE VALIDATION -----
if "age_touched" not in st.session_state:
    st.session_state.age_touched = False

def mark_age_touched():
    st.session_state.age_touched = True

with col2:

```



```

age = st.number_input(
    "Age",
    min_value=0,
    max_value=100,
    value=0,
    step=1,
    key="age_input",
    on_change=mark_age_touched
)

# ----- AGE GATE -----
age_valid = True

if st.session_state.age_touched and age < 18:
    age_valid = False

    st.markdown(
        "<span style='color:#ff4b4b;'>⚠ Age must be 18 or above to receive  

recommendations.</span>",
        unsafe_allow_html=True
    )

with col3:
    mood = st.selectbox(
        "Current Mood",
        ["Happy", "Sad", "Angry", "Calm", "Energetic"]
    )

# -----
# Single Music Preference
# -----
st.subheader("🎵 Music Preferences")

colp1, colp2 = st.columns(2)

with colp1:
    genre_pref = st.selectbox(
        "Preferred Genre",
        ["Bollywood", "Hindi Pop", "Ghazal"]
    )

with colp2:
    era_pref = st.selectbox(
        "Preferred Vibe",
        ["60s songs", "90s songs", "Energetic songs", "calming songs", "Classical songs"]
    )

# -----

```

```

# Smartwatch Inputs
# -----
st.header("🕒 Smartwatch Data")

col3, col4 = st.columns(2)
with col3:
    hrv = st.slider("HR (bpm)", 20, 200, 90)
with col4:
    stress = st.slider("Stress Level", 0, 100, 40)

hrv_n = (hrv - 20) / 180
stress_n = stress / 100

# -----
# Psychological Inputs (RANDOM QUESTIONS)
# -----
st.header("🧠 Psychological Inputs")

# ----- Question Banks -----
TIPI_ALL = [
    "Q1. I see myself as extraverted, enthusiastic.",
    "Q2. I see myself as critical, quarrelsome.",
    "Q3. I see myself as dependable, self-disciplined.",
    "Q4. I see myself as anxious, easily upset.",
    "Q5. I see myself as open to new experiences, complex.",
    "Q6. I see myself as reserved, quiet.",
    "Q7. I see myself as sympathetic, warm.",
    "Q8. I see myself as disorganized, careless.",
    "Q9. I see myself as calm, emotionally stable.",
    "Q10. I see myself as conventional, uncreative."
]

DASS_ALL = [
    "Q1. I found it hard to wind down.",
    "Q2. I was aware of dryness of my mouth.",
    "Q3. I couldn't seem to experience any positive feeling at all.",
    "Q4. I experienced breathing difficulty.",
    "Q5. I found it difficult to work up the initiative to do things.",
    "Q6. I tended to over-react to situations.",
    "Q7. I experienced trembling.",
    "Q8. I felt that I was using a lot of nervous energy.",
    "Q9. I was worried about situations in which I might panic.",
    "Q10. I felt that I had nothing to look forward to.",
    "Q11. I found myself getting agitated.",
    "Q12. I found it difficult to relax.",
    "Q13. I felt down-hearted and blue.",
    "Q14. I was intolerant of anything that kept me from getting on with what I was doing.",
    "Q15. I felt I was close to panic.",

```

"Q16. I was unable to become enthusiastic about anything.",
 "Q17. I felt I wasn't worth much as a person.",
 "Q18. I felt that I was rather touchy.",
 "Q19. I was aware of the action of my heart.",
 "Q20. I felt scared without any good reason.",
 "Q21. I felt that life was meaningless."

]

WHOQOL_ALL = [

"Q1. How would you rate your quality of life?",
 "Q2. How satisfied are you with your health?",
 "Q3. To what extent do you feel that pain prevents you from doing what you need to do?",
 "Q4. How much do you need any medical treatment to function in your daily life?",
 "Q5. How much do you enjoy life?",
 "Q6. To what extent do you feel your life to be meaningful?",
 "Q7. How well are you able to concentrate?",
 "Q8. How safe do you feel in your daily life?",
 "Q9. How healthy is your physical environment?",
 "Q10. Do you have enough energy for everyday life?",
 "Q11. Are you able to accept your bodily appearance?",
 "Q12. Have you enough money to meet your needs?",
 "Q13. How available is the information that you need in your daily life?",
 "Q14. To what extent do you have the opportunity for leisure activities?",
 "Q15. How satisfied are you with your sleep?",
 "Q16. How satisfied are you with your ability to perform daily living activities?",
 "Q17. How satisfied are you with your capacity for work?",
 "Q18. How satisfied are you with yourself?",
 "Q19. How satisfied are you with your personal relationships?",
 "Q20. How satisfied are you with your sex life?",
 "Q21. How satisfied are you with the support from your friends?",
 "Q22. How satisfied are you with your living conditions?",
 "Q23. How satisfied are you with access to health services?",
 "Q24. How satisfied are you with your transport?",
 "Q25. How well are you able to get around?",
 "Q26. Are you satisfied with your environment?"

]

----- TIPI -----

st.subheader("🌱 TIPI (Big Five)")

st.caption("1 = Disagree strongly | 7 = Agree strongly")

tipi = []

for q in TIPI_ALL:

tipi.append(st.slider(q, 1, 7, 4))

----- DASS-21 -----

st.subheader("🗨️ DASS-21")

st.caption("0 = Did not apply | 3 = Applied very much")

dass = []

```

for q in DASS_ALL:
    dass.append(st.slider(q, 0, 3, 1))

# ----- WHOQOL-BREF -----
st.subheader("🌍 WHOQOL-BREF")
st.caption("1 = Very poor | 5 = Very good")
whoqol = []
for q in WHOQOL_ALL:
    whoqol.append(st.slider(q, 1, 5, 3))

# =====
# ### NEW: SCORE CALCULATION (STANDARDIZED & MANUAL-CORRECT)
# =====
# ----- TIPI (Big Five) -----
# Reverse coding: items 2,4,6,8,10
def rev_tipi(x):
    return 8 - x

tipi_scored = tipi.copy()
for idx in [1, 3, 5, 7, 9]: # 0-based indices
    tipi_scored[idx] = rev_tipi(tipi_scored[idx])

# Big Five traits (average of two items each)
extraversion = (tipi_scored[0] + tipi_scored[5]) / 2
agreeableness = (tipi_scored[1] + tipi_scored[6]) / 2
conscientiousness = (tipi_scored[2] + tipi_scored[7]) / 2
emotional_stability = (tipi_scored[3] + tipi_scored[8]) / 2
openness = (tipi_scored[4] + tipi_scored[9]) / 2

# ----- DASS-21 -----
# Item indices are 0-based
dep_items = [2, 4, 9, 12, 15, 16, 20]
anx_items = [1, 3, 6, 8, 14, 18, 19]
str_items = [0, 5, 7, 10, 11, 13, 17]

depression = sum(dass[i] for i in dep_items) * 2
anxiety = sum(dass[i] for i in anx_items) * 2
stress_s = sum(dass[i] for i in str_items) * 2

dass_total = depression + anxiety + stress_s

def get_dass_mood(depression, stress_s, anxiety):
    if depression >= 20:
        return "Sad"
    elif stress_s >= 26:
        return "Angry"
    elif anxiety >= 16 and depression < 10:
        return "Energetic"

```

```

elif depression < 10 and anxiety < 10 and stress_s < 10:
    return "Calm"
else:
    return "Happy"

def final_mood(user_mood, dass_mood, weight_user=0.7):
    if user_mood == dass_mood:
        return user_mood
    return user_mood if random.random() < weight_user else dass_mood

# ----- Mood Mapping Based on DASS-21 -----
dass_mood = get_dass_mood(depression, stress_s, anxiety)
mood_state = final_mood(mood, dass_mood, weight_user=0.7)

# ----- WHOQOL-BREF -----
# Reverse score items: Q3, Q4, Q26 → indices 2,3,25
def rev_whoqol(x):
    return 6 - x

whoqol_scored = whoqol.copy()
for idx in [2, 3, 25]:
    whoqol_scored[idx] = rev_whoqol(whoqol_scored[idx])

# Domain raw scores
physical_raw = sum(whoqol_scored[i] for i in [2, 3, 9, 14, 15, 16, 17])
psych_raw    = sum(whoqol_scored[i] for i in [4, 5, 6, 10, 18, 25])
social_raw   = sum(whoqol_scored[i] for i in [19, 20, 21])
env_raw      = sum(whoqol_scored[i] for i in [7, 8, 11, 12, 13, 22, 23, 24])

# Domain means
physical_mean = physical_raw / 7
psych_mean    = psych_raw / 6
social_mean   = social_raw / 3
env_mean      = env_raw / 8

# Transform to 0–100 scale
physical_qol = (physical_mean - 4) * (100 / 16)
psych_qol    = (psych_mean - 4) * (100 / 16)
social_qol   = (social_mean - 4) * (100 / 16)
env_qol      = (env_mean - 4) * (100 / 16)

# -----
# RL Setup
# -----
q_dir = "QTables"
os.makedirs(q_dir, exist_ok=True)

user_file = os.path.join(q_dir, f"{name.lower()}_q.csv")

```

```

global_file = os.path.join(q_dir, "global_q.csv")

def load_q(path, fallback=None):
    if os.path.exists(path):
        return np.loadtxt(path, delimiter=",")
    elif fallback is not None and os.path.exists(fallback):
        return np.loadtxt(fallback, delimiter=",") # new user starts from global average
    return np.zeros((100, num_songs))

personal_q = load_q(user_file, fallback=global_file) # ← new user gets global as starting point
global_q = load_q(global_file)

# ---- Session number tracking ---- # ← ADD FROM HERE
if os.path.exists(feedback_file := os.path.join(q_dir, f"{name.lower()}_feedback.csv")):
    _existing = pd.read_csv(feedback_file)
    if "session_number" in _existing.columns:
        st.session_state["session_number"] = int(_existing["session_number"].max()) + 1
    else:
        st.session_state["session_number"] = 3 # existing user, no column yet = session 2+
else:
    st.session_state["session_number"] = 1 # brand new user = session 1

# ---- SAFETY: Resize Q-tables if dataset size changed ----
if personal_q.shape[1] != num_songs:
    new_q = np.zeros((100, num_songs))
    min_cols = min(personal_q.shape[1], num_songs)
    new_q[:, :min_cols] = personal_q[:, :min_cols]
    personal_q = new_q

if global_q.shape[1] != num_songs:
    new_q = np.zeros((100, num_songs))
    min_cols = min(global_q.shape[1], num_songs)
    new_q[:, :min_cols] = global_q[:, :min_cols]
    global_q = new_q

# -----
# Session State Init for Control
# -----
if "recs" not in st.session_state:
    st.session_state["recs"] = []
if "got_rec" not in st.session_state:
    st.session_state["got_rec"] = False
if "feedback_count" not in st.session_state:
    st.session_state["feedback_count"] = 0
if "pool" not in st.session_state:
    st.session_state["pool"] = pd.DataFrame()
if "session_number" not in st.session_state:

```

```

st.session_state["session_number"] = 1

def psychology_bias(row, mood_state, extraversion, openness,
                    depression, psych_qol, physical_qol, social_qol):

    score = 0.0
    genre = str(row.get("genre", "")).lower()

    # Mood softness factor (0–1)
    mood_factor = 0.3 if mood_state in ["Sad", "Angry"] else 0.1

    #① Mood influence (SOFTENED)
    if any(g in genre for g in ["slow", "soft", "ghazal", "classical"]):
        score += mood_factor * (depression / 42)

    #② Personality influence (scaled)
    personality_strength = (extraversion + openness) / 14
    if any(g in genre for g in ["dance", "pop", "bollywood"]):
        score += 0.2 * personality_strength

    #③ Physical QOL influence
    if physical_qol < 40:
        score += 0.1 * (1 - physical_qol / 100)

    #④ Social QOL influence
    if social_qol < 40:
        score += 0.05 * (1 - social_qol / 100)

    return score

def get_user_state(mood_state, stress, depression):
    """
    Convert user psychological + physiological condition into RL state (0–99)
    """
    state = 0

    # Mood-based bins
    if mood_state == "Sad":
        state += 10
    elif mood_state == "Angry":
        state += 20
    elif mood_state == "Energetic":
        state += 30
    elif mood_state == "Calm":
        state += 40
    else: # Happy
        state += 50

```

```
# Stress contribution
```

```
if stress > 70:
```

```
    state += 10
```

```
elif stress > 40:
```

```
    state += 5
```

```
# Depression contribution
```

```
if depression > 20:
```

```
    state += 5
```

```
return min(state, 99)
```

```
FALLBACK_WEIGHTS = np.array([0.40, 0.28, 0.18, 0.09, 0.03, 0.02])
```

```
def get_final_weights(feedback_file, fallback_weights):
```

```
    MIN_SAMPLES = 20
```

```
    if not os.path.exists(feedback_file):
```

```
        return fallback_weights
```

```
    feedback_df = pd.read_csv(feedback_file)
```

```
    required_cols = ["rnn_score", "ncf_score", "personal_q",  
                    "pref_bias", "physio_fit", "psy_bias", "rating"]
```

```
    if not all(c in feedback_df.columns for c in required_cols):
```

```
        return fallback_weights
```

```
    if len(feedback_df) < MIN_SAMPLES:
```

```
        return fallback_weights
```

```
    try:
```

```
        from sklearn.linear_model import Ridge
```

```
        X = feedback_df[["rnn_score", "ncf_score", "personal_q",  
                        "pref_bias", "physio_fit", "psy_bias"]].values
```

```
        y = feedback_df["rating"].values
```

```
        model = Ridge(alpha=1.0)
```

```
        model.fit(X, y)
```

```
        clipped = np.clip(model.coef_, 0.01, None)
```

```
        return clipped / clipped.sum()
```

```
    except Exception:
```

```
        return fallback_weights
```

```
MOOD_MAP = {
```

```
    "Sad": 0,
```

```
    "Angry": 1,
```

```
    "Energetic": 2,
```

```
    "Calm": 3,
```

```
    "Happy": 4
```

```
}
```

```
mood_id = MOOD_MAP[mood_state]
```

```
stress_n = stress / 100.0
```

```
hrv_n = (hrv - 20) / 180.0
```



```

# -----
# Recommendations (Single Click)
# -----
st.header("🎵 Recommendations")

get_recs_btn = st.button("🎧 Get Recommendations", disabled=not age_valid)

if get_recs_btn and not st.session_state["got_recs"]:
    # Mark that we have already generated recommendations for this user
    st.session_state["got_recs"] = True

# 1 Apply USER PREFERENCE first (UPDATED)
pool = df.copy()

# — Helper: sub-filter by audio features safely —
def safe_filter(base_pool, condition, min_size=5):
    filtered = base_pool[condition]
    return filtered if len(filtered) >= min_size else base_pool

# — STEP 1: VIBE FILTER —
if era_pref == "Classical songs":
    classical_pool = pool[pool["genre"].str.contains(
        "raga|classical|hindustani|carnatic|traditional music|contemporary classical|chamber
music",
        case=False, na=False
    )]
    if len(classical_pool) >= 1:
        pool = classical_pool
else:
    if era_pref == "60s songs":
        era_pool = pool[(pool["year"] >= 1960) & (pool["year"] <= 1969)]
        if len(era_pool) >= 20:
            pool = era_pool
    elif era_pref == "90s songs":
        era_pool = pool[(pool["year"] >= 1990) & (pool["year"] <= 1999)]
        if len(era_pool) >= 20:
            pool = era_pool

# — STEP 2: GENRE FILTER —
genre_pool = pool[pool["genre"].str.contains(
    genre_pref.lower(), case=False, na=False
)]
if len(genre_pool) >= 10:
    pool = genre_pool

# — STEP 3: VIBE AUDIO FILTER (consistent with mood filters) —
if era_pref == "Energetic songs":

```

```

    # High energy + high valence regardless of genre
    vibe_pool = pool[
        (pool["energy"] > 0.60) & (pool["valence"] > 0.45)
    ]
    if len(vibe_pool) >= 10:
        pool = vibe_pool

elif era_pref == "calming songs":
    # Low energy + moderate-high valence → peaceful
    vibe_pool = pool[
        (pool["energy"] < 0.50) & (pool["valence"] > 0.30)
    ]
    if len(vibe_pool) >= 10:
        pool = vibe_pool

# — STEP 4: MOOD AUDIO SUB-FILTER —————
if mood_state == "Sad":
    pool = safe_filter(pool,
        (pool["energy"] < 0.50) & (pool["valence"] < 0.55))
elif mood_state == "Happy":
    pool = safe_filter(pool,
        (pool["valence"] > 0.50) & (pool["energy"] > 0.45))
elif mood_state == "Angry":
    pool = safe_filter(pool,
        (pool["energy"] > 0.50) & (pool["valence"] < 0.60))
elif mood_state == "Calm":
    pool = safe_filter(pool,
        (pool["energy"] < 0.55) & (pool["valence"] > 0.30))
elif mood_state == "Energetic":
    pool = safe_filter(pool,
        (pool["energy"] > 0.55) & (pool["valence"] > 0.45))

# — STEP 5: HRV SUB-FILTER —————
if hrv > 100:
    pool = safe_filter(pool, pool["energy"] < 0.55)
elif hrv < 50:
    pool = safe_filter(pool, pool["energy"] > 0.35)

# — STEP 6: STRESS SUB-FILTER —————
if stress > 70:
    pool = safe_filter(pool,
        (pool["energy"] < 0.55) & (pool["valence"] > 0.25))
elif stress < 30:
    pool = safe_filter(pool, pool["energy"] > 0.35)

# — STEP 7: PSYCHOLOGICAL SUB-FILTERS —————
if depression >= 20:
    pool = safe_filter(pool, pool["valence"] > 0.30)

```

```

if anxiety >= 16:
    pool = safe_filter(pool, pool["energy"] < 0.65)
if extraversion > 5:
    pool = safe_filter(pool, pool["energy"] > 0.40)
if physical_qol < 25:
    pool = safe_filter(pool, pool["energy"] < 0.60)
if social_qol < 25:
    pool = safe_filter(pool, pool["valence"] > 0.35)

```

— STEP 8: REMIX FILTER —————

```

clean_pool = pool[~pool["song"].str.contains(
    "trap mix|remix|mashup|lo-fi mix", case=False, na=False
)]
if len(clean_pool) >= 5:
    pool = clean_pool

```

```

def preference_bias(row, genre_pref, era_pref):
    score = 0.0
    g = str(row.get("genre", "")).lower()
    y = row.get("year", None)

```

```

# 🎵 Genre preference
if genre_pref.lower() in g:
    score += 0.3

```

```

# ⚡ Vibe preference
if era_pref == "Energetic songs" and any(k in g for k in ["dance", "upbeat", "pop"]):
    score += 0.2

```

```

if era_pref == "calming songs" and any(k in g for k in ["soft", "slow", "instrumental"]):
    score += 0.2

```

```

if era_pref == "Classical songs" and any(k in g for k in ["raga", "classical", "hindustani",
"caranatic", "traditional music", "contemporary classical", "chamber music"]):
    score += 0.2

```

```

# 🕒 Era preference
if y is not None:
    if era_pref == "60s songs" and 1960 <= y <= 1969:
        score += 0.25
    elif era_pref == "90s songs" and 1990 <= y <= 1999:
        score += 0.25

```

```

return score

```

🗝 KNN Candidate Generation (CONTENT SIMILARITY)

```

# pool["knn_bonus"] = 0.0

# # Safe reference selection
# ref_song_id = random.randint(0, num_songs - 1)
# ref_vec = knn_features[ref_song_id].reshape(1, -1)

# _, indices = knn_model.kneighbors(ref_vec)
# knn_song_ids = set(indices[0])

# pool["knn_bonus"] = pool["song_id"].apply(
# lambda x: 0.2 if x in knn_song_ids else 0.0
# )

# -----
#  Add Mood–HRV–Stress Fit Score (AFTER filtering)
# -----
pool = pool.copy() # safety
pool["physio_fit"] = pool.apply(
lambda row: mood_physiology_fit(row, mood_state, hrv, stress),
axis=1
)

#4  RL-based ranking
state = get_user_state(mood_state, stress, depression)

pool = pool.copy()

pool["personal_q"] = pool["song_id"].apply(
lambda a: personal_q[state, a]
)

pool["global_q"] = pool["song_id"].apply(
lambda a: global_q[state, a]
)

pool["psy_bias"] = pool.apply(
lambda row: psychology_bias(
    row,
    mood_state,
    extraversion,
    openness,
    depression,
    psych_qol,
    physical_qol,
    social_qol
),
axis=1
)

```

```

# ---- RNN Score ----
SEQ_LEN = 10 # MUST match training

last_seq = random.sample(range(NUM_SONGS_TRAINED), min(SEQ_LEN,
NUM_SONGS_TRAINED))
shared_seq = torch.tensor(
    [random.sample(range(NUM_SONGS_TRAINED), min(SEQ_LEN,
NUM_SONGS_TRAINED))],
    dtype=torch.long
)

# --- Proper psychological normalization (MATCH TRAINING FORMAT) ---

# TIPI: use overall personality mean (1-7 → 0-1)
tipi_mean = np.mean(tipi) # your 10 TIPI sliders
tipi_n = (tipi_mean - 1) / 6.0

# WHOQOL: convert back to 1-5 style scale before normalization
# psych_mean is already 1-5 domain mean → perfect for model
whoql_n = (psych_mean - 1) / 4.0

# DASS: your sliders are 0-3 each → use average instead of total
dass_mean = np.mean(dass) # 0-3 scale
dass_n = dass_mean / 3.0 # normalize to 0-1

# Mood normalization (training likely used 0-1 encoded mood)
mood_n = MOOD_MAP[mood_state] / 4.0

# FINAL RNN CONTEXT (6 FEATURES — REQUIRED)
context = torch.tensor(
    [[mood_n, stress_n, hrv_n, tipi_n, whoql_n, dass_n]],
    dtype=torch.float32
)

with torch.no_grad():
    logits = rnn_model(shared_seq, torch.tensor([0]), torch.tensor([0]), context)
    probs = torch.softmax(logits, dim=1).squeeze()

pool["rnn_score"] = pool["song_id"].apply(
    lambda i: probs[i % NUM_SONGS_TRAINED].item()
)

# ---- NCF Score ----
user_hash = hash(name) % NUM_USERS_TRAINED
user_id = torch.tensor([user_hash], dtype=torch.long)
mood_t = torch.tensor([MOOD_MAP[mood_state]], dtype=torch.long)
stress_t = torch.tensor([stress_n], dtype=torch.float32)
hrv_t = torch.tensor([hrv_n], dtype=torch.float32)

```

```

def ncf_score(song_id):
    genre_id = int(df.loc[song_id, "genre_id"])
    vibe_id = int(df.loc[song_id, "vibe_id"])

    genre_id = max(0, min(genre_id, NUM_GENRES_TRAINED - 1))
    vibe_id = max(0, min(vibe_id, NUM_VIBES_TRAINED - 1))

    song_id_safe = song_id % NUM_SONGS_TRAINED

    song_id_t = torch.tensor([song_id_safe], dtype=torch.long)
    genre_id_t = torch.tensor([genre_id], dtype=torch.long)
    vibe_id_t = torch.tensor([vibe_id], dtype=torch.long)
    mood_t_t = torch.tensor([MOOD_MAP[mood_state]], dtype=torch.long)
    stress_t_t = torch.tensor([stress_n], dtype=torch.float32)
    hrv_t_t = torch.tensor([hrv_n], dtype=torch.float32)
    tipi_t = torch.tensor([tipi_n], dtype=torch.float32)
    dass_t = torch.tensor([dass_n], dtype=torch.float32)
    whoql_t = torch.tensor([whoql_n], dtype=torch.float32)
    pop_t = torch.tensor([0.5], dtype=torch.float32)

    with torch.no_grad():
        score = ncf_model(user_id, song_id_t, genre_id_t, vibe_id_t, mood_t_t, stress_t_t,
            hrv_t_t, tipi_t, dass_t, whoql_t, pop_t)

    return score.item()

pool["ncf_score"] = pool["song_id"].apply(ncf_score)

pool["pref_bias"] = pool.apply(
    lambda r: preference_bias(r, genre_pref, era_pref),
    axis=1
)

weights = get_final_weights(
    os.path.join(q_dir, f"{name.lower()}_feedback.csv"),
    FALLBACK_WEIGHTS
)

def safe_normalize(col):
    min_v = col.min()
    max_v = col.max()
    if max_v - min_v < 1e-6:
        return np.zeros_like(col) + 0.5
    return (col - min_v) / (max_v - min_v)

# — STEP 1: Diversity penalty —————
genre_counts = pool["genre"].value_counts()

```

```
pool["diversity_penalty"] = pool["genre"].map(
    lambda g: np.log1p(genre_counts[g])
) * 0.01
```

```
# — STEP 2: Exploration bonus —————
```

```
global_feedback_file = os.path.join(q_dir, "global_session_feedback.csv")
if os.path.exists(global_feedback_file):
    global_feedback_df = pd.read_csv(global_feedback_file)
    if "song_id" in global_feedback_df.columns:
        song_counts = np.bincount(
            global_feedback_df["song_id"].astype(int),
            minlength=num_songs
        )
    else:
        song_counts = np.zeros(num_songs)
else:
    song_counts = np.zeros(num_songs)
```

```
exploration_bonus = 0.1 / (1 + song_counts)
pool["exploration_bonus"] = pool["song_id"].map(lambda i: exploration_bonus[i])
```

```
# — STEP 3: Normalize (exploration_bonus exists now) —————
```

```
cols = [
    "personal_q", "global_q", "psy_bias",
    "physio_fit", "pref_bias",
    "rnn_score", "ncf_score", "exploration_bonus"
]
for c in cols:
    pool[c] = safe_normalize(pool[c])
```

```
# — STEP 4: Final score —————
```

```
pool["final_score"] = (
    weights[0] * pool["rnn_score"] +
    weights[1] * pool["ncf_score"] +
    weights[2] * pool["personal_q"] +
    weights[3] * pool["pref_bias"] +
    weights[4] * pool["physio_fit"] +
    weights[5] * pool["psy_bias"] +
    0.05 * pool["exploration_bonus"]
) - pool["diversity_penalty"]
```

```
weights_sum = 1
pool["final_score"] /= weights_sum
```

```
# Sort by score first
```

```
pool_sorted = pool.sort_values("final_score", ascending=False)
```

```
# Take top 15 candidates
```

```

top_candidates = pool_sorted.head(15)

# Exploration probability
epsilon = 0.15

n_available = len(top_candidates)
n_pick = min(5, n_available)

if random.random() < epsilon:
    chosen = top_candidates.sample(n_pick, replace=False)
else:
    chosen = top_candidates.head(n_pick)

# Pad with next best songs if fewer than 5 available
if len(chosen) < 5:
    remaining = pool[~pool["song_id"].isin(chosen["song_id"])]
    extras = remaining.nlargest(5 - len(chosen), "final_score")
    chosen = pd.concat([chosen, extras], ignore_index=True)

# Save recommendations
st.session_state["pool"] = pool
st.session_state["recs"] = (
    chosen[["song_id", "song", "artist", "genre"]]
    .to_dict("records")
)

if get_recs_btn:
    st.session_state["feedback_count"] = 0 # reset feedback count for this set

# -----
# Show Recommendations
# -----
if "recs" in st.session_state and st.session_state["recs"]:

    # Initialize interaction flag
    if "spotify_touched" not in st.session_state:
        st.session_state.spotify_touched = {}

    if "pending_feedback_song" not in st.session_state:
        st.session_state.pending_feedback_song = None

    if "lock_warning_song" not in st.session_state:
        st.session_state.lock_warning_song = None

    if "song_touched" not in st.session_state:
        st.session_state.song_touched = {}

    for i, s in enumerate(st.session_state["recs"]):

```



```

if i not in st.session_state.spotify_touched:
    st.session_state.spotify_touched[i] = False

st.subheader(f"{i+1}. {s['song']} – {s['artist']}")

# ----- UNIQUE FEEDBACK FLAG -----
flag_key = f"fb_done_{i}_{s['song_id']}"

if flag_key not in st.session_state:
    st.session_state[flag_key] = False

# ----- RATING -----
rating = st.radio(
    "Rate this song (1 = Strongly dislike, 5 = Strongly like)",
    [1, 2, 3, 4, 5],
    horizontal=True,
    key=f"rate_{i}_{s['song_id']}"
)

# ----- FEEDBACK BUTTON -----
feedback_btn = st.button(
    f"Submit Feedback for Song {i+1}",
    key=f"fb_{i}_{s['song_id']}",
    disabled=st.session_state[flag_key]
)

if feedback_btn:
    st.session_state[flag_key] = True
    # st.success("✅ Feedback recorded")

# 🔓 AUTO RELEASE LOCK
if st.session_state.pending_feedback_song == i:
    st.session_state.pending_feedback_song = None
    st.session_state.lock_warning_song = None

# ----- interaction marker -----
def mark_song_touched(idx):
    st.session_state.song_touched[idx] = True

# =====
# 🎧 SPOTIFY BUTTON + SEQUENTIAL VALIDATION
# =====

# Initialize interaction flag
# if "spotify_touched" not in st.session_state:
#     st.session_state.spotify_touched = {}

```

```

# if "pending_feedback_song" not in st.session_state:
#     st.session_state.pending_feedback_song = None

# if "lock_warning_song" not in st.session_state:
#     st.session_state.lock_warning_song = None

# if i not in st.session_state.spotify_touched:
#     st.session_state.spotify_touched[i] = False

spotify_url = spotify_link(s["song"], s["artist"])

spotify_key = f"spotify_{i}_{s['song_id']}"

# 🎧 Spotify button - HTML approach (ALWAYS works)
if st.button(f"🎧 Open Song {i+1} in Spotify", key=spotify_key):

    allow_open = True
    locked = st.session_state.pending_feedback_song

    # 🚫 If another song is locked → block opening
    if locked is not None and locked != i:

        locked_song = st.session_state["recs"][locked]
        locked_flag = f"fb_done_{locked}_{locked_song['song_id']}"

        if not st.session_state.get(locked_flag, False):
            st.session_state.lock_warning_song = locked
            allow_open = False

    # ✅ Allowed → open & lock THIS song
    if allow_open:

        st.session_state.pending_feedback_song = i
        st.session_state.lock_warning_song = None

        from streamlit.components.v1 import html
        html(f"""
        <script>
            window.open('{spotify_url}', '_blank');
        </script>
        """, height=0)

    # ----- Warning under previously opened song -----
    if st.session_state.lock_warning_song == i:

        warn_song = st.session_state["recs"][i]
        warn_flag = f"fb_done_{i}_{warn_song['song_id']}"

```

```

if not st.session_state.get(warn_flag, False):

    st.markdown(
        "<span style='color:#ff4b4b;'>⚠ Submit feedback before opening another
song</span>",
        unsafe_allow_html=True
    )

# ----- SUBMIT FEEDBACK -----
if feedback_btn:

    song_action = s["song_id"]
    # Improved RL reward function
    reward_map = {1: -1.0, 2: -0.5, 3: 0.0, 4: 0.5, 5: 1.0}
    reward = reward_map.get(rating, 0)

    feedback_file = os.path.join(q_dir, f'{name.lower()}_feedback.csv')

    new_entry = {
        "song_id": song_action,
        "song": s["song"],
        "artist": s["artist"],
        "rating": rating,
        "hrv": hrv,
        "stress": stress,
        "session_number": st.session_state["session_number"],
        "extraversion": extraversion,
        "agreeableness": agreeableness,
        "conscientiousness": conscientiousness,
        "emotional_stability": emotional_stability,
        "openness": openness,
        "depression": depression,
        "anxiety": anxiety,
        "stress_score": stress_s,
        "physical_qol": physical_qol,
        "psych_qol": psych_qol,
        "social_qol": social_qol,
        "env_qol": env_qol,
        "age": age,
        "mood_id": mood_id,
        "rnn_score": float(st.session_state["pool"].loc[st.session_state["pool"]]["song_id"]
== song_action, "rnn_score"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,
        "ncf_score": float(st.session_state["pool"].loc[st.session_state["pool"]]["song_id"]
== song_action, "ncf_score"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,
        "personal_q": float(st.session_state["pool"].loc[st.session_state["pool"]]["song_id"]
== song_action, "personal_q"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,
        "pref_bias": float(st.session_state["pool"].loc[st.session_state["pool"]]["song_id"] ==
song_action, "pref_bias"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,

```

```

        "physio_fit": float(st.session_state["pool"].loc[st.session_state["pool"]["song_id"] ==
song_action, "physio_fit"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,
        "psy_bias": float(st.session_state["pool"].loc[st.session_state["pool"]["song_id"]
== song_action, "psy_bias"].values[0]) if len(st.session_state["pool"]) > 0 else 0.0,
    }

    if os.path.exists(feedback_file):
        feedback_df = pd.read_csv(feedback_file)
    else:
        feedback_df = pd.DataFrame(columns=new_entry.keys())

    feedback_df = pd.concat(
        [feedback_df, pd.DataFrame([new_entry])],
        ignore_index=True
    )
    feedback_df.to_csv(feedback_file, index=False)

    current_state = get_user_state(mood_state, stress, depression)

    update_q(personal_q, current_state, song_action, reward, current_state)
    update_q(global_q, current_state, song_action, reward, current_state)

    np.savetxt(user_file, personal_q, delimiter=",")
    np.savetxt(global_file, global_q, delimiter=",")

    st.session_state[flag_key] = True
    st.session_state["feedback_count"] += 1

    st.success("Feedback recorded")

# -----
# Footer
# -----
st.markdown("---")

if "session_finished" not in st.session_state:
    st.session_state.session_finished = False

# -----
# Finish Session Button
# -----
if st.session_state["recs"]:

    if st.button("✅ Finish Listening Session"):
        st.session_state.session_finished = True

# -----
# PARTICIPANT RL FEEDBACK (SESSION END)

```

```

# -----
if st.session_state.session_finished:

    st.header("★ Overall System Feedback ")

    comfort = st.slider(
        "Q1. How comfortable did you feel using the system? (1 = Very uncomfortable, 10 = Extremely comfortable)",
        1, 10, 5
    )

    satisfaction = st.slider(
        "Q2. How satisfied are you with the recommendations? (1 = Very dissatisfied, 10 = Extremely satisfied)",
        1, 10, 5
    )

    mood_alignment = st.slider(
        "Q3. How well did songs match your mood? (1 = Not at all, 10 = Perfectly matched)",
        1, 10, 5
    )

    experience = st.slider(
        "Q4. Rate your overall experience? (1 = Poor, 10 = Excellent)",
        1, 10, 5
    )

    continue_use = st.radio(
        "Q5. Would you like to continue using this system?",
        ["Yes", "No"]
    )

    overall_btn = st.button("Submit the Feedback")

    if overall_btn:

        entry = {
            "user": name.lower(),
            "comfort": comfort,
            "satisfaction": satisfaction,
            "mood_alignment": mood_alignment,
            "experience": experience,
            "continue": continue_use,
            "avg_song_rating": np.mean([
                st.session_state.get(
                    f"rate_{i}_{s['song_id']}", 3
                )
                for i, s in enumerate(st.session_state["recs"])
            ])
        }

```

```

    ]),
    "mood_state": mood_state,
    "stress": stress,
    "hrv": hrv
}

# ----- PERSONAL CSV -----
personal_file = os.path.join(
    q_dir,
    f"{name.lower()}_session_feedback.csv"
)

if os.path.exists(personal_file):
    df_personal = pd.read_csv(personal_file)
else:
    df_personal = pd.DataFrame(columns=entry.keys())

df_personal = pd.concat(
    [df_personal, pd.DataFrame([entry])],
    ignore_index=True
)

df_personal.to_csv(personal_file, index=False)

# ----- GLOBAL CSV -----
global_file = os.path.join(
    q_dir,
    "global_session_feedback.csv"
)

if os.path.exists(global_file):
    df_global = pd.read_csv(global_file)
else:
    df_global = pd.DataFrame(columns=entry.keys())

df_global = pd.concat(
    [df_global, pd.DataFrame([entry])],
    ignore_index=True
)

df_global.to_csv(global_file, index=False)

st.success("Thank you for participating!")

# Optional reset
st.session_state.session_finished = False

```